

Linux Environment Setup and Simple Shell Development

- 2520030561

2520030106

2520030183

2520030480

Aim

To set up the Linux environment and develop a simple Linux shell using C programming.

Theory

A shell is a command-line interface that allows users to interact with the Linux operating system. It accepts user commands and executes them using system calls.

Important system calls:

- `fork()` – Creates a child process.
- `execvp()` – Executes a command.
- `wait()` – Waits for the child process to complete.

A Makefile is used to automate compilation.

Procedure

1. Install Ubuntu or WSL.
2. Install GCC.
3. Create a project folder.
4. Initialize a Git repository.
5. Create the C source file.
6. Create a Makefile.
7. Compile the program.

8. Run the program.
9. Enter Linux commands.
10. Type exit to close the shell.

Commands Used

```
D:\Documents\Sem 2-1\OS\2520030561_Skill>wsl
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill$ nano Task2.c
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill$ |
```

Task2.c

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <string.h>
```

```
#include <unistd.h>
```

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
#define MAX_INPUT 1024
```

```
#define MAX_ARGS 64
```

```
int main()
```

```
{
```

```
    char input[MAX_INPUT];
```

```
    char *args[MAX_ARGS];
```

```
    printf("=====\n");
```

```
    printf("    Simple Linux Shell\n");
```

```
    printf("Type 'exit' to quit the shell\n");
```

```
    printf("=====\n");
```

```
    while (1)
```

```

{
    // Display shell prompt
    printf("\nMyShell> ");
    fflush(stdout);

    // Read user input
    if (fgets(input, sizeof(input), stdin) == NULL)
    {
        printf("\n");
        break;
    }

    // Remove newline character
    input[strcspn(input, "\n")] = '\0';

    // Ignore empty input
    if (strlen(input) == 0)
        continue;

    // Exit condition
    if (strcmp(input, "exit") == 0)
    {
        printf("Exiting MyShell...\n");
        break;
    }

    // Split input into arguments
    int i = 0;
    args[i] = strtok(input, " ");

    while (args[i] != NULL && i < MAX_ARGS - 1)

```

```

{
    i++;
    args[i] = strtok(NULL, " ");
}

args[i] = NULL;

// Create child process
pid_t pid = fork();

if (pid < 0)
{
    perror("Fork Failed");
    continue;
}

// Child Process
if (pid == 0)
{
    printf("\n----- Child Process ----- \n");
    printf("Child PID : %d\n", getpid());
    printf("Parent PID : %d\n\n", getppid());

    execvp(args[0], args);

    perror("Command Execution Failed");
    exit(EXIT_FAILURE);
}

// Parent Process
else

```

```

{
    printf("\n----- Parent Process ----- \n");
    printf("Parent PID : %d\n", getpid());
    printf("Waiting for Child Process...\n");

    wait(NULL);

    printf("Child Process Completed.\n");
}
}

return 0;
}

```

```

prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill$ nano Makefile
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill$ |

```

Makefile

CC = gcc

CFLAGS = -Wall

TARGET = Task2

SOURCE = Task2.c

all:

\$(CC) \$(CFLAGS) \$(SOURCE) -o \$(TARGET)

run:

./\$(TARGET)

clean:

rm -f \$(TARGET)

```
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill$ make
gcc -Wall Task2.c -o Task2
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill$ make run
./Task2
```

```
=====
      Simple Linux Shell
      Type 'exit' to quit the shell
=====
```

```
MyShell> |
```

```
MyShell> pwd
/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill
```

```
MyShell> ls
Makefile Task2 Task2.c Week1Skill Week1Skill.c Week2Skill.c
```

```
MyShell> uname
Linux
```

```
MyShell> date
Sun Aug 2 13:31:42 UTC 2026
```

```
MyShell> whoami
prem
```

```
MyShell> lscpu
Architecture:          x86_64
  CPU op-mode(s):      32-bit, 64-bit
  Address sizes:        39 bits physical, 48 bits virtual
  Byte Order:           Little Endian
CPU(s):                12
  On-line CPU(s) list: 0-11
Vendor ID:             GenuineIntel
  Model name:           Intel(R) Core(TM) 5 120U
    CPU family:         6
    Model:              186
```

```
MyShell> exit
Exiting Shell...
```

```
prem@premsbook4:/mnt/d/Documents/Sem 2-1/OS/2520030561_Skill$ |
```

Observation

- Linux environment was configured successfully.
- GCC compiler was installed and tested.
- Git repository was created.
- The Makefile compiled the program.
- The shell accepted user commands.
- Commands were executed using fork() and execvp().
- The shell exited successfully using the exit command.

Result

- The Linux environment was successfully configured, and a simple shell was developed. The shell executed Linux commands correctly and terminated when the `exit` command was entered.